



Boas práticas de Programação

Métodos Avançados de Programação

hiago.fernandes@servidor.uepb.edu.br

O que é um código limpo?

- ✨ **Definição:**

Código limpo é aquele que é **fácil de entender, modificar e manter**, mesmo por alguém que não o escreveu.

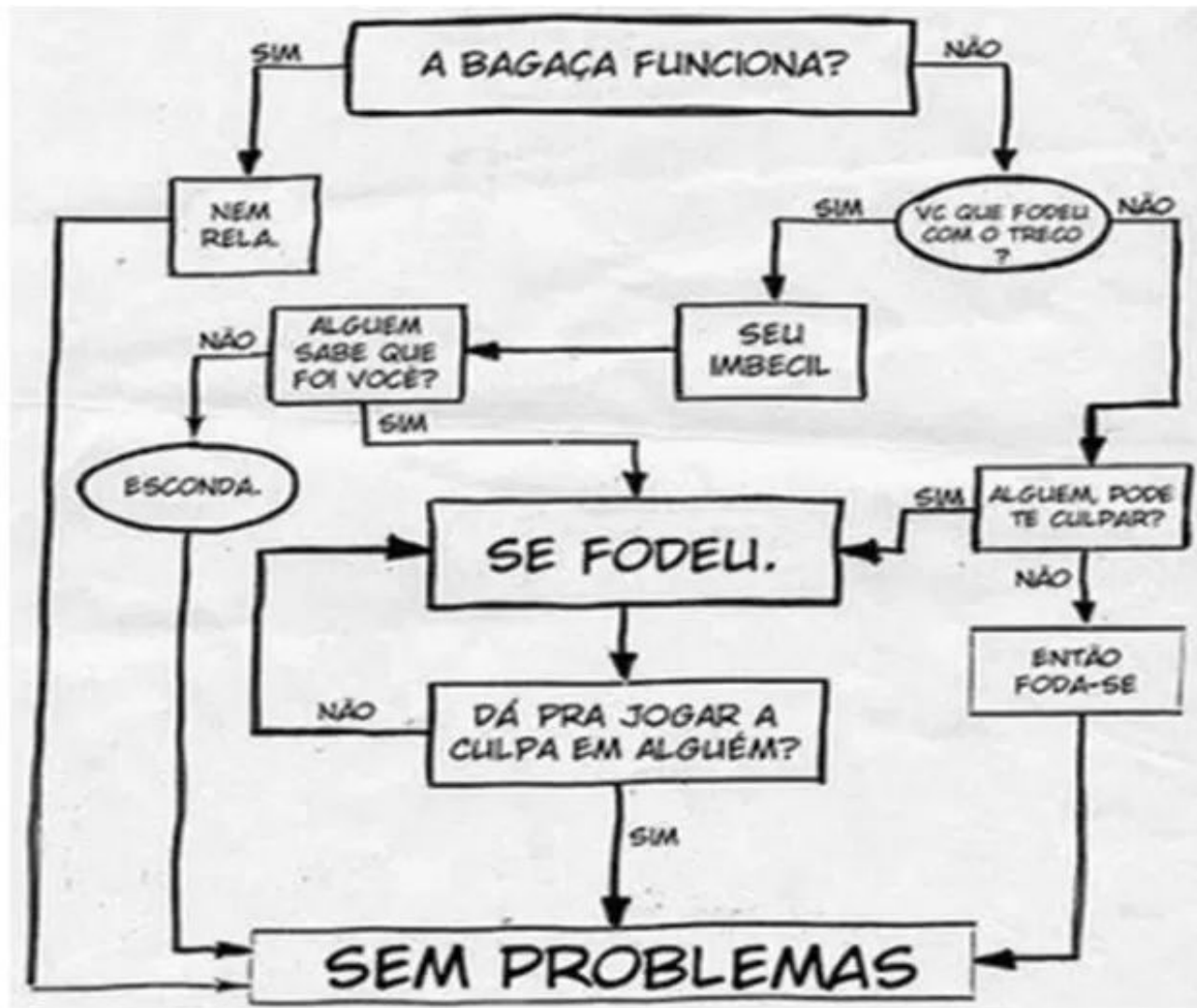
- 🔑 **Características:**

- Clareza nos nomes, métodos e responsabilidades
- Simplicidade na solução
- Ausência de duplicação
- Estrutura consistente

- 🚀 **Por que é crucial?**

- Facilita a **manutenção** e reduz erros
- Aumenta a **produtividade da equipe**
- Prolonga a **vida útil do software**
- Torna o sistema mais **flexível a mudanças**

Cuidado!!



Cuidado!!



Seu código é o pior que já vi



Mas funciona

Code Smell?



What is that smell???
Did you write that code?

O código é ruim? Tenta refatorar...

Quando você termina de consertar o código mas agora ele não funciona mais



Cuidado!!

"A única medida verdadeira de qualidade de código é a contagem de 'WTFs' por minuto durante uma revisão de código."



O Problema: O Custo do Código Ruim

Visual sugerido:

- Gráfico de linhas simples
 - Eixo X: **Tempo**
 - Eixo Y: **Custo de Manutenção**
 - Linha vermelha (Código Sujo): curva exponencial crescente
 - Linha verde (Código Limpo): quase reta, crescimento mínimo

Cuidado!!

Tópicos

- Código ruim **atrasa projetos**
- **Aumenta a frustração** da equipe
- Gera **bugs e instabilidade**
- **Lei de LeBlanc:** *Mais tarde = Nunca* → “depois arrumamos” quase nunca acontece

O que é Código Limpo?

✨ Características

- **Elegante e Eficiente**
Faz uma coisa bem feita.
- **Fácil de Ler e Entender**
Parece uma prosa bem escrita.
- **Simples e Direto**
Evita complexidade desnecessária.
- **Fácil de Manter e Modificar**
Foi escrito pensando no próximo desenvolvedor.
- **Completo em Testes**
É confiável e robusto.

🏕️ Regra do Escoteiro

"Deixe a área do acampamento (o código) mais limpa do que você a encontrou."

Pilar 1: Nomes Significativos

☀️ Regra de Ouro

"O nome de uma variável, função ou classe deve responder a todas as grandes questões. Deve dizer por que existe, o que faz e como é usado."

```
//Use nomes que revelem a intenção
// Ruim
int d;

// Bom
int diasDesdeACriacao;

//Faça distinções significativas
// Ruim
produto1, produto2;

// Bom
produtoOrigem, produtoDestino;

//Use nomes pronunciáveis e buscáveis
// Ruim
String genymdhms;

// Bom
String gerarDataHoraMinutoSegundo;
```

Pilar 1: Nomes Significativos

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class Clientes extends Pessoas {
5
6      public Clientes(String nome, String cpf, String email) {
7          super(nome, cpf, email);
8      }
9
10
11 }
12
```



Pilar 1: Nomes Significativos

```
public Funcionario marcaPonto(String cpf) {  
    for (int i = 0; i < funcionarios.size(); i++) {  
        Funcionario funcionario = funcionarios.get(i);  
  
        if (cpf.equals(funcionario.getCpf())) {  
            return funcionario;  
        }  
    }  
    return null;  
}
```



Pilar 1: Nomes Significativos



```
public class Aluno {
```

```
    private String nome;  
    private String matricula;  
    private String email;  
    private int tel;  
    private Notas notas;  
    public int x = 0;
```

```
    public Aluno(String nome, String matricula, String email, int tel ) {  
        this.nome = nome;  
        this.matricula = matricula;  
        this.email = email;  
        this.tel = tel;  
    }
```

```
    ...
```

```
}
```



Pilar 1: Nomes Significativos



```
public class Aluno {  
    //Atributos  
    private String nome;  
    private String matricula;  
    private String email;  
    private int tel;  
    private Notas notas;  
    public int x = 0; //variavel para verificar se notas foram cadastradas  
  
    //metodo construtor  
    public Aluno(String nome, String matricula, String email, int tel ) {  
        this.nome = nome;  
        this.matricula = matricula;  
        this.email = email;  
        this.tel = tel;  
    }  
  
    ...  
}
```

Pilar 2: Funções pequenas e coesas

Funções pequenas e coesas

Uma função deve fazer apenas **uma coisa** e fazê-la bem.

Evite funções longas, com múltiplas responsabilidades.

Esse princípio segue o **SRP (Single Responsibility Principle)**, garantindo clareza, legibilidade e manutenção facilitada do código.

Pilar 2: Funções pequenas e coesas

```
public Boolean alugarLivro(Acervo acervo, Usuario usuario){

    Scanner ler = new Scanner(System.in);

    System.out.println("Digite o nome do livro que deseja alugar: ");
    String livroAlugar = ler.next();
    ler.close();
    for(int i = 0; i< acervo.getAutores().size(); i++) {
        for(int a = 0; a<acervo.getAutores().get(a).getLivros().size(); a++) {
            if(livroAlugar == acervo.getAutores().get(a).getLivros().get(a).getNomeLivro() &&
acervo.getAutores().get(a).getLivros().get(a).getQuantidadeLivro() >
acervo.getAutores().get(a).getLivros().get(a).getQuantidadeLivroAlugado()) {

                this.setLivro(acervo.getAutores().get(a).getLivros().get(a));

acervo.getAutores().get(a).getLivros().get(a).setQuantidadeLivroAlugado(acervo.getAutores().get(a).getLivros().get(a).
getQuantidadeLivroAlugado()+ 1);
                usuario.setAlugueis(this);
                return true;
            }
        }
    }
}
```



Pilar 2: Funções pequenas e coesas

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        Scanner leitor = new Scanner(System.in);
        Scanner leitorB = new Scanner(System.in);
        ArrayList<Cliente> clientes = new ArrayList<Cliente>();
        ArrayList<Livro> livros = new ArrayList<Livro>();

        while (true) {
            int opcao = 0;
            System.out.println("Digite os números para as seguintes opções");
            System.out.println("\n1: Adicionar cliente.");
            System.out.println("\n2: Adicionar um livro no estante.");
            System.out.println("\n3: Ver clientes cadastrados.");
            System.out.println("\n4: Ver livros cadastrados.");
            System.out.println("\n5: Remover um cliente.");
            System.out.println("\n6: Remover livro da estante.");
            opcao = leitor.nextInt();
            switch (opcao) {
                case 1:
                    char controle = 's';
                    do {
                        System.out.println("\nDigite o nome: ");
                        String nome = leitorB.nextLine();
                        System.out.println("Digite o CPF: ");
                        String cpf = leitorB.nextLine();
                        System.out.println("Digite o email: ");
                        String email = leitorB.nextLine();
                        Cliente cliente = new Cliente(nome, cpf, email);
                        clientes.add(cliente);
                        System.out.println("Deseja cadastrar outra pessoa? 's' ou 'S' para sim e 'n' ou 'N' para não.");
                        controle = leitorB.next().charAt(0);
                    } while (controle == 's' || controle == 'S');
                    break;

                case 2:
                    char controle2 = 's';
                    do {
                        System.out.println("\nDigite o nome do livro: ");
                        String nomeLivro = leitorB.nextLine();
                        System.out.println("Digite o gênero: ");
                        String genero = leitorB.nextLine();
                        System.out.println("Digite o Autor: ");
                        String autor = leitorB.nextLine();
                        Livro livro = new Livro(nomeLivro, genero, autor);
                        livros.add(livro);
                        System.out.println("Deseja adicionar outro livro? Se sim Digite 's' ou 'S', se não, digite 'n' ou 'N'");
                        controle2 = leitorB.next().charAt(0);
                    } while (controle2 == 's' || controle2 == 'S');
                    break;

                case 3:
                    if (clientes.isEmpty()) {
                        System.out.println("Ainda não há clientes cadastrados.");
                    } else {
                        for (Cliente i : clientes) {
                            System.out.println(i);
                        }
                    }
                    break;

                case 4:
                    if (livros.isEmpty()) {
                        System.out.println("Ainda não há livros cadastrados.");
                    } else {
                        for (Livro i : livros) {
                            System.out.println(i);
                        }
                    }
                    break;

                case 5:
                    int removedor;
                    System.out.println("\nEscolha um usuário a ser removido a partir da posição 0: ");
                    for (Cliente i : clientes) {
                        System.out.println(i);
                    }
                    removedor = leitorB.nextInt();
                    System.out.println(clientes.get(removedor));
                    System.out.println("Você esse for o usuário que deseja remover Digite 's' ou 'S', se não, digite 'n' ou 'N'");
                    char confirmar;
                    confirmar = leitorB.next().charAt(0);
                    if (confirmar == 's' || confirmar == 'S') {
                        clientes.remove(removedor);
                        System.out.println("Usuário removido com sucesso");
                    }
                    break;
                    System.out.println("\nOperação cancelada.");
                    break;

                case 6:
                    int removedor2;
                    System.out.println("\nEscolha um livro a ser removido a partir da posição 0: ");
                    for (Livro i : livros) {
                        System.out.println(i);
                    }
                    removedor2 = leitorB.nextInt();
                    removedor2 = removedor2 - 1;
                    System.out.println(livros.get(removedor2));
                    System.out.println("Você esse for o livro que deseja remover Digite 's' ou 'S'");
                    char confirmar2;
                    confirmar2 = leitorB.next().charAt(0);
                    if (confirmar2 == 's' || confirmar2 == 'S') {
                        livros.remove(removedor2);
                        System.out.println("Livro removido com sucesso");
                    }
                    break;
                    System.out.println("\nOperação cancelada.");
                    break;

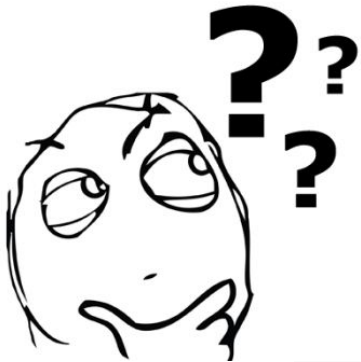
                default:
                    System.out.println("\nOpção inválida");
                    break;
            }
        }
    }
}
```



Pilar 2: Funções pequenas e coesas



```
public void cadastrarAdmin(){  
  
    Admin admin = new Admin();  
    Scanner ler = new Scanner(System.in);  
  
    System.out.println("Digite o Login:");  
    admin.setLoginAdmin(ler.next());  
  
    System.out.println("Digite a senha:");  
    admin.setSenhaAdmin(ler.next());  
    ler.close();  
  
    adicionarAdmin(admin);  
  
}
```



Pilar 2: Funções pequenas e coesas



```
public Boolean adicionarLivro(Acervo acervo) {

    Livro livro = new Livro();
    Scanner ler = new Scanner(System.in);
    boolean existe = true;
    System.out.println("#####");
    System.out.println("                Digite o nome do Livro:                ");
    livro.setNomeLivro(ler.next());
    System.out.println("                Digite o Gênero:                ");
    livro.setGeneroLivro(ler.next());
    System.out.println("                Digite o ano da publicação:                ");
    livro.setAnoPublicacao(ler.next());
    System.out.println("                Digite a quantidade de livros:                ");
    livro.setQuantidadeLivro(ler.nextInt());
    System.out.println("#####");
    livro.setQuantidadeLivroAlugado(0);
    String inputNomeAutor;
    while(existe){

        System.out.println("                Digite o nome do autor");
        inputNomeAutor = ler.next();
        for(int i = 0; i < acervo.getAutores().size(); i++){
            if(acervo.getAutores().get(i).getNomeAutor() == inputNomeAutor) {
                existe = false;
                acervo.getAutores().get(i).adicionarLivro(livro);
                System.out.println("                Livro adicionado");
                ler.close();
                return true;
            }

        }
        System.out.println("                Autor não existe.");
    }
    ler.close();
    return false;
}
```

Pilar 3: Clareza e simplicidade

O código deve ser fácil de entender, mesmo por quem não o escreveu.

Evite **gambiarrras**, duplicação desnecessária e complexidade artificial.

Código limpo lê-se como um texto bem escrito.

Pilar 3: Clareza e simplicidade

```
public static Funcionario senhaFuncionario(int senhafuncionario) {  
    Funcionario funcionario = null;  
    if (fichaFuncionarios.size() > 0) {  
        for (Funcionario auxiliar : fichaFuncionarios) {  
            if (auxiliar.getSenhafuncionario() == senhafuncionario) {  
                funcionario = auxiliar;  
                return funcionario;  
            }  
        }  
    }  
  
    ComponenteVisual.senhaInvalida();  
  
    Menu.menuPrincipal();  
    return funcionario;  
}
```



Pilar 3: Clareza e simplicidade



```
public class Cofre {  
    public void executar(Scanner scanner) {  
        int[] numeros = new int[10];  
        int soma = 0;  
  
        System.out.println("\n== Cofre Misterioso ==");  
        for (int i = 0; i < 10; i++) {  
            System.out.print("Número " + (i + 1) + ": ");  
            numeros[i] = scanner.nextInt();  
            soma += numeros[i];  
        }  
  
        System.out.println("A soma total é: " + soma);  
    }  
}
```



Pilar 3: Clareza e simplicidade

```
public void RemoveAluno(String matricula) {  
    int indice = -1;  
    int local = 0;  
    for (Aluno i : this.alunos) {  
        indice += 1;  
        if (i.getMatricula().equals(matricula)) {  
            local = indice;  
        }  
    }  
    alunos.remove(local);  
}
```



Pilar 4: Tratamento consistente de erros

O código deve lidar com erros de forma clara e previsível.

Não ignore falhas silenciosamente, pois isso dificulta a manutenção e a detecção de problemas.

Mantenha um padrão único para captura, registro (logging) e propagação de erros.

- Evite comportamentos ocultos ou silenciosos.
- Comunique falhas de maneira transparente.
- Use padrões consistentes de tratamento, como exceções bem definidas e registro adequado (logging).
- Um tratamento de erros previsível torna o sistema mais **confiável** e **manutenível**.

Pilar 4: Tratamento consistente de erros

Ruim

- Ignora a falha silenciosamente
- Não informa o usuário
- Dificulta depuração

java

```
try {  
    int resultado = 10 / 0;  
} catch (Exception e) {  
    // nada acontece  
}
```

Pilar 4: Tratamento consistente de erros

Bom

- Registra o erro
- Comunica de forma clara
- Mantém o sistema previsível

java

```
try {  
    int resultado = 10 / 0;  
} catch (ArithmeticException e) {  
    System.err.println("Erro: divisão por zero não é permitida.");  
    e.printStackTrace();  
}
```

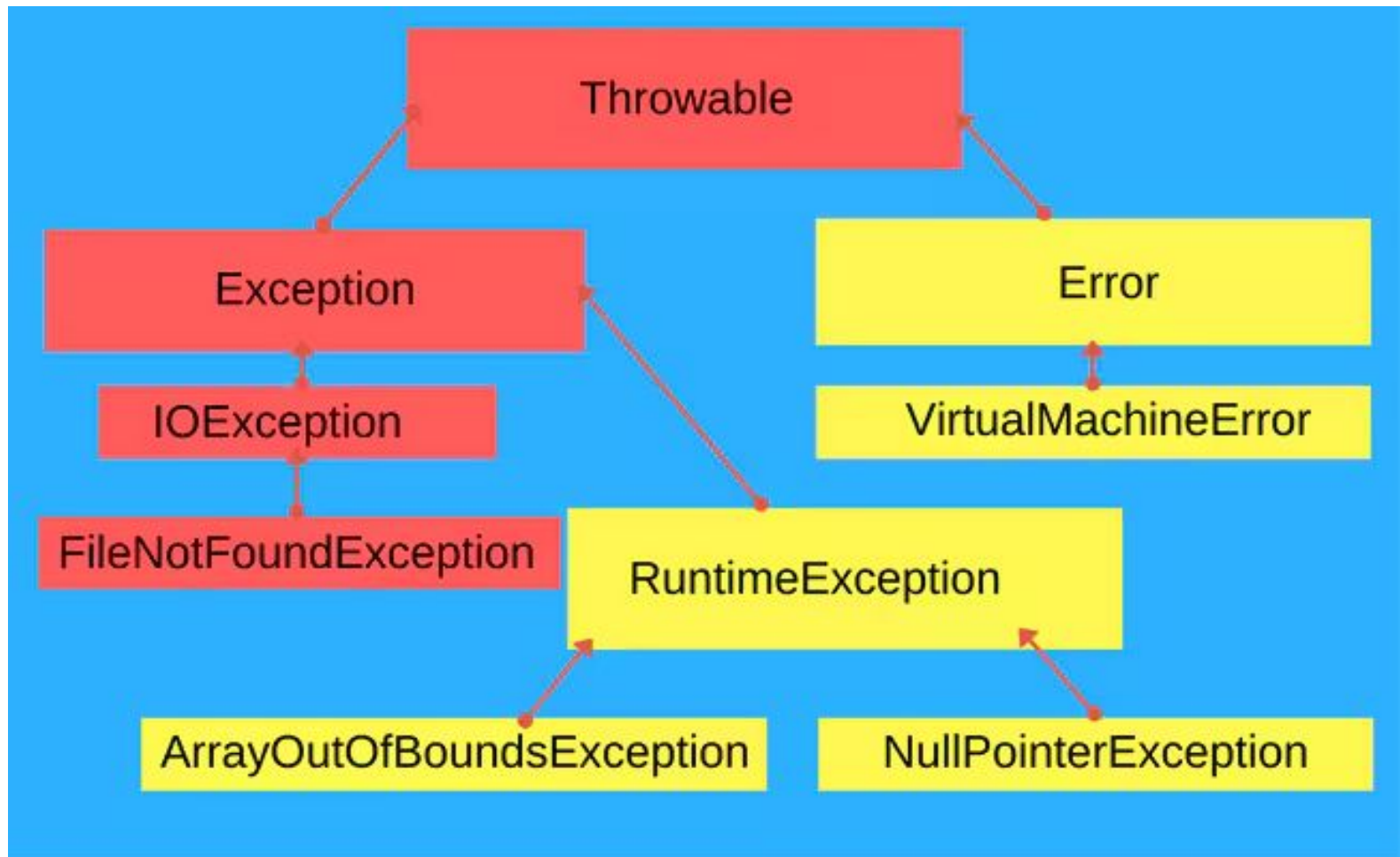
Como tratar as Exceptions?

- Crie exceções de domínio claras: `SaldoInsuficienteException`, `PedidoInvalidoException`.
- Organize uma **hierarquia** (ex.: `DominioException` ← específicas).
- **Não** use exceções genéricas do sistema na borda pública do seu domínio.



```
public class DominioException extends RuntimeException {  
    public DominioException(String msg) {  
        super(msg);  
    }  
}  
  
public class SaldoInsuficienteException extends DominioException {  
  
    public SaldoInsuficienteException(BigDecimal saldo, BigDecimal valor) {  
        super("Saldo " + saldo + " insuficiente para " + valor);  
    }  
}
```

Hierarquia de Exceptions



Hierarquia de Exceptions

◆ 1. Exceptions verificadas (Checked Exceptions)

Exemplos: `IOException`, `FileNotFoundException`

- O compilador **obriga** você a tratá-las (com `try/catch` ou `throws`).
- Se não forem tratadas, o programa **não compila**.
- Se forem lançadas e não capturadas, interrompem a execução.
- Se forem capturadas, o programa continua normalmente após o `catch` .

■ Resumo:

Paralisam a execução **somente se não forem tratadas**.

Hierarquia de Exceptions

◆ 2. Exceções em tempo de execução (Unchecked / Runtime Exceptions)

Exemplos: `NullPointerException`, `ArrayIndexOutOfBoundsException`

- O compilador **não obriga** o tratamento.
- Se ocorrerem e **não forem tratadas**, interrompem o programa imediatamente.
- Mesmo que não sejam obrigatórias, **podem ser capturadas** com `try/catch` para evitar a parada do sistema.

Resumo:

Paralisam a execução se **não tratadas**, mas são **opcionais** de capturar.

Hierarquia de Exceptions

◆ 3. Erros (Error)

Exemplos: `VirtualMachineError`, `OutOfMemoryError`, `StackOverflowError`

- São falhas **graves** da JVM.
- Normalmente **não devem ser capturadas** (mesmo que seja possível).
- Quase sempre **interrompem a execução** do programa, pois indicam condições irreversíveis.

■ Resumo:

Praticamente sempre **param o programa**, mesmo que você tente capturar.

Hierarquia de Exceptions

⚙ Em resumo:

Tipo	Tratamento obrigatório	Interrompe o programa?	Deve ser tratada?
Checked Exception	✅ Sim	❌ Não (se tratada) / ✅ Sim (se não tratada)	✅ Sim
Runtime Exception	❌ Não	✅ Sim (se ocorrer)	⚠ Depende do caso
Error	❌ Não	✅ Sim (quase sempre)	❌ Não (evitar tratar)

Pilar 5: Formatação e organização

A clareza do código também depende da sua **estrutura visual**.

Uma boa formatação facilita a leitura, a revisão e a manutenção.

Use **indentação consistente**, **espaços adequados** e **nomes bem alinhados** para melhorar a legibilidade.

Evite blocos longos e confusos, e mantenha o código organizado por seções lógicas.

Um código bem formatado transmite **cuidado**, **profissionalismo** e **colaboração** dentro da equipe.

Pilar 5: Formatação e organização



```
class Emp {  
    User u; Pub p; Date data; int prazo;  
    Emp(User u, Pub p, int pr) { this.u=u; this.p=p; this.prazo=pr; this.data=new Date(); }  
}
```



```
class User {  
    String nome, tipo;  
    User(String n, String t) { nome=n; tipo=t; }  
}
```

Pilar 5: Formatação e organização

```
import java.util.Objects;

public class Pessoa {

    private String nome;
    private int idade;
    private String cidade;

    public Pessoa(String nome, int idade, String cidade) {
        this.nome = nome;
        this.idade = idade;
        this.cidade = cidade;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    //Outros métodos get() e set()...

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Pessoa)) return false;
        Pessoa pessoa = (Pessoa) o;
        return idade == pessoa.idade &&
            Objects.equals(nome, pessoa.nome) &&
            Objects.equals(cidade, pessoa.cidade);
    }

    @Override
    public int hashCode() {
        return Objects.hash(nome, idade, cidade);
    }

    @Override
    public String toString() {
        return "Pessoa{" +
            "nome=" + nome + '\'' +
            ", idade=" + idade +
            ", cidade=" + cidade + '\'' +
            '}';
    }
}
```

Pilar 6: Evitar duplicação (DRY – Don't Repeat Yourself)

Cada pedaço de lógica deve existir em **um único lugar** no código.

Repetir instruções ou blocos semelhantes torna a manutenção mais difícil e aumenta o risco de erros.

Se um comportamento é reutilizado, **extraia para um método, classe ou constante**.

A duplicação quebra a coesão e espalha responsabilidades, dificultando correções futuras.

Código limpo é **conciso, reutilizável e centralizado** — escreva uma vez, use várias.

Pilar 6: Evitar duplicação (DRY – Don't Repeat Yourself)



```
public class Calculadora {  
  
    public void somarNumerosPequenos() {  
        int a = 2;  
        int b = 3;  
        int resultado = a + b;  
        System.out.println("Resultado: " + resultado);  
    }  
  
    public void somarNumerosGrandes() {  
        int a = 100;  
        int b = 200;  
        int resultado = a + b;  
        System.out.println("Resultado: " + resultado);  
    }  
  
}
```

Pilar 6: Evitar duplicação (DRY – Don't Repeat Yourself)



```
public class Calculadora {  
  
    public void somar(int a, int b) {  
        int resultado = a + b;  
        System.out.println("Resultado: " + resultado);  
    }  
  
}
```

- Agora, a lógica está centralizada em um único método.
- O código é mais limpo, reutilizável e fácil de manter.
- Se for preciso mudar algo, basta editar um único lugar.

Pilar 7: Comentário com propósito

Comentários devem existir para **explicar o “porquê”** o código faz algo, e não o **“como”**.

Um código limpo e bem escrito **dispensa comentários desnecessários**, pois se explica por si mesmo. Evite comentários redundantes ou óbvios, que apenas repetem o que já está no código.

Use comentários para:

- Explicar **decisões de design** incomuns.
- Destacar **situações excepcionais** ou **regras de negócio específicas**.
- Documentar **motivos de escolha**, e não a execução passo a passo.

Comentários devem **agregar valor e contexto**, não mascarar um código confuso.

Pilar 7: Comentário com propósito

```
# Função para somar dois números
def somar(a, b):
    return a + b # Retorna a soma

# Testando a função
resultado = somar(2, 3) # Soma 2 e 3
print(resultado) # Imprime o resultado
```

Pilar 7: Comentário com propósito



```
def somar(a, b):  
    return a + b
```

```
resultado = somar(2, 3)  
print(resultado)
```

Pilar 7: Comentário com propósito

```
// Ruim  
// Verifica se o usuário está logado  
if (usuario != null)  
  
// Bom  
if (usuarioLogado != null)
```

Pilar 8: Limite de responsabilidade

Um código bem escrito evita “classes faz-tudo”.

Cada classe/método deve ter **uma responsabilidade clara e um único motivo para mudar**.

- **Separe tarefas em unidades menores e coesas** (cálculo, persistência, validação, notificação).
- Prefira **métodos curtos** com nomes que indiquem claramente o que fazem.
- Evite **efeitos colaterais misturados** (ex.: calcular + salvar + enviar e-mail no mesmo método).
- Isso melhora **manutenção, reutilização e testabilidade**.

Exemplo (separação de responsabilidades):

`PedidoService` (orquestra) → `CalculadoraDeTotal` (regra de negócio) → `PedidoRepository` (persistência) → `EmailSender` (notificação)

Pilar 8: Limite de responsabilidade

```
// Ruim
public class ProcessadorDePagamento
{
    public void ProcessarPagamento(Pedido pedido)
    {
        // código para processar o pagamento
        // código para atualizar o estoque
        // código para enviar notificação
        // ...
    }
}

// Bom
public class ProcessadorDePagamento
{
    public void ProcessarPagamento(Pedido pedido)
    {
        // código para processar o pagamento
    }
}

public class Estoque
{
    public void AtualizarEstoque(Pedido pedido)
    {
        // código para atualizar o estoque
    }
}

public class Notificador
{
    public void EnviarNotificacao(Pedido pedido)
    {
        // código para enviar notificação
    }
}
```

Pilar 9: Efeitos Colaterais

Uma função deve fazer o que promete **sem mexer em outras coisas escondidas**.

Isso deixa o código **mais previsível e mais fácil de testar**.

- Evite alterar **variáveis globais** ou **objetos compartilhados** sem deixar claro.
- Não misture **cálculo** com **ações** (salvar no banco, enviar e-mail, escrever arquivo).
- Prefira funções que **recebem dados e devolvem um resultado**, sem mudar o “mundo” ao redor.
- Se precisar mudar algo (BD, e-mail, log), faça isso **em outro método** e de forma explícita.

Exemplo:

`calcularTotal(itens)` → só calcula

`salvarPedido(pedido)` → só salva

`enviarEmailConfirmacao()` → só envia

Pilar 9: Efeitos Colaterais

A ideia de **efeito colateral** é: *você chama um método esperando UMA coisa, mas ele também faz OUTRA coisa escondida (muda um dado fora dele).*

```
class Cliente {  
    double saldo;  
}  
  
class DescontoService {  
  
    // Parece que só calcula o desconto...  
    static double aplicarDesconto(double valor, Cliente cliente) {  
        cliente.saldo -= 10;           // efeito colateral: mexe no cliente!  
        return valor - 10;  
    }  
}
```

Pilar 9: Efeitos Colaterais

Como corrigir isso?

```
class Cliente {  
    double saldo;  
}  
  
class DescontoService {  
  
    // Parece que só calcula o desconto...  
    static double aplicarDesconto(double valor, Cliente cliente) {  
        cliente.saldo -= 10;           // efeito colateral: mexe no cliente!  
        return valor - 10;  
    }  
}
```

Pilar 9: Efeitos Colaterais

```
class Cliente {  
    double saldo;  
}  
  
class DescontoService {  
    static double calcularValorComDesconto(double valor) {  
        return valor - 10;  
    }  
  
    static void debitarSaldo(Cliente cliente, double valor) {  
        cliente.saldo -= valor;           // alteração explícita  
    }  
}
```

Pilar 10: Refatoração Contínua

Refatorar é **melhorar o código por dentro** sem mudar o comportamento do sistema.

A ideia é **não deixar a bagunça acumular**: pequenas melhorias o tempo todo.

- **Simplifique** trechos complexos (menos “gambiarras”, mais clareza).
- **Elimine duplicação** (mesma lógica em vários lugares).
- **Melhore a estrutura** (nomes, responsabilidades, coesão e acoplamento).
- Faça refatorações **pequenas e frequentes**, de preferência junto com a tarefa do dia.

Boas práticas:

- ✓ Refatore com **testes passando** (ou crie testes antes)
- ✓ “Boy Scout Rule”: *deixe o código um pouco melhor do que encontrou*

Pilar 10: Refatoração Contínua

```
class CalculadoraFrete {  
  
    double freteSP(double pesoKg) {  
        double base = 10;  
        return base + (pesoKg * 2);  
    }  
  
    double freteRJ(double pesoKg) {  
        double base = 10;  
        return base + (pesoKg * 2);  
    }  
}
```

Pilar 10: Refatoração Contínua

```
class CalculadoraFrete {  
    double calcularFrete(double pesoKg) {  
        double base = 10;  
        return base + (pesoKg * 2);  
    }  
}
```

Pilar 10: Refatoração Contínua

```
class SistemaAcademico {  
  
    void cadastrarAluno(String nome, String matricula, String email) {  
        // valida  
        if (nome == null || nome.isEmpty()) System.out.println("Nome inválido");  
        if (matricula == null || matricula.isEmpty()) System.out.println("Matrícula inválida");  
        if (email == null || !email.contains("@")) System.out.println("Email inválido");  
  
        // formata  
        nome = nome.trim();  
        matricula = matricula.trim();  
        email = email.trim().toLowerCase();  
  
        // cadastra (simulado)  
        System.out.println("UEPB: aluno cadastrado -> " + nome + " (" + matricula + ")");  
  
        // envia aviso (simulado)  
        System.out.println("Email enviado para: " + email);  
    }  
}
```

Pilar 10: Refatoração Contínua

```
class SistemaAcademico {  
  
    void cadastrarAluno(String nome, String matricula, String email) {  
        if (!dadosValidos(nome, matricula, email)) return;  
  
        nome = limpar(nome);  
        matricula = limpar(matricula);  
        email = limpar(email).toLowerCase();  
  
        cadastrarNoSistema(nome, matricula);  
        enviarConfirmacao(email);  
    }  
  
    boolean dadosValidos(String nome, String matricula, String email) {  
        if (nome == null || nome.isEmpty()) return false;  
        if (matricula == null || matricula.isEmpty()) return false;  
        if (email == null || !email.contains("@")) return false;  
        return true;  
    }  
  
    String limpar(String texto) {  
        return texto.trim();  
    }  
  
    void cadastrarNoSistema(String nome, String matricula) {  
        System.out.println("UEPB: aluno cadastrado -> " + nome + " (" + matricula + ")");  
    }  
  
    void enviarConfirmacao(String email) {  
        System.out.println("Email enviado para: " + email);  
    }  
}
```

Questões

```
import java.util.*;

public class Gerenciador {

    public static List<String> dados = new ArrayList<>();

    public void p(String t, int n, boolean s, String tipo) {

        if (t == "TURMA_EXTRA") {
            System.out.println("Pulando...");
            return;
        }

        try {

            if (tipo.equals("F") && n >= 7) {
                String msg = "Turma: " + t + " Status: OK";
                System.out.println(msg);
                dados.add(msg);

                if (s) {
                    enviarDadosParaOExterno(t, "Aprovado");
                }
            } else {
                if (n < 0) {
                    throw new RuntimeException("Nota inválida");
                }
                String msg = "Turma: " + t + " Status: REPROVADO";
                dados.add(msg);
            }
        } catch (Exception e) {
            // Silencia o erro para o programa não parar
        }

        private void enviarDadosParaOExterno(String id, String st) {
            // Envia notificação...
        }
    }
}
```

Analise o código acima e, com base nos princípios de "Código Limpo" (Clean Code), assinale a alternativa que identifica corretamente a principal falha de design e a respectiva boa prática que deveria ser aplicada para corrigi-la:

- A) O nome da função `p` é aceitável por ser curto e economizar memória, mas o uso de `int n` viola a regra de tipos primitivos. A solução seria criar uma classe `Nota` para encapsular o valor e garantir que ele nunca seja negativo, eliminando o `try-catch`.
- B) A função `p` viola o princípio de que "Funções devem fazer apenas uma coisa". Ela valida a nota, gera mensagens de log, decide sobre notificações e manipula uma lista global. A solução correta seria decompor `p` em funções menores, como `isAprovada()`, `registrarLog()` e `notificarResultado()`.
- C) O principal erro é o uso do parâmetro `boolean s`, conhecido como *Flag Argument*. Segundo o Clean Code, uma função nunca deve receber um booleano. A solução seria renomear a função para `processarComNotificacao` e remover todos os outros parâmetros, deixando apenas o booleano.
- D) O código falha ao usar `System.out.println` dentro de um bloco `try`, o que causa um efeito colateral de I/O. A solução de acordo com a refatoração é mover todos os textos para constantes globais e substituir o `if-else` por um comando `switch` para tratar o tipo de turma.
- E) A função é considerada limpa pois utiliza nomes de variáveis curtos (`t`, `n`, `s`), o que facilita a leitura rápida. O único erro é a comparação de Strings com `.equals()`, que deveria ser substituída por `==` para garantir que estamos comparando o mesmo objeto na memória.

Questões

```
import java.util.*;

public class Gerenciador {

    public static List<String> dados = new ArrayList<>();

    public void p(String t, int n, boolean s, String tipo) {

        if (t == "TURMA_EXTRA") {
            System.out.println("Pulando...");
            return;
        }

        try {

            if (tipo.equals("F") && n >= 7) {
                String msg = "Turma: " + t + " Status: OK";
                System.out.println(msg);
                dados.add(msg);

                if (s) {
                    enviarDadosParaOExterno(t, "Aprovado");
                }
            } else {
                if (n < 0) {
                    throw new RuntimeException("Nota inválida");
                }
                String msg = "Turma: " + t + " Status: REPROVADO";
                dados.add(msg);
            }
        } catch (Exception e) {
            // Silencia o erro para o programa não parar
        }

        private void enviarDadosParaOExterno(String id, String st) {
            // Envia notificação...
        }
    }
}
```

Alternativa Correta: B

- **Por que a B está correta?** A função `p` é uma "Função Monstro" em miniatura. Ela decide se deve processar, valida a regra, formata string, salva em log e envia notificação. No Clean Code, se uma função tem o conectivo "e" (valida e loga e notifica), ela deve ser dividida.
- **Por que as outras estão incorretas?**
 - **A:** Nomes curtos como `p` são péssimos (nomes devem ser claros/pronunciáveis). Criar uma classe `Nota` é bom, mas o motivo dado para a falha de `p` é secundário.
 - **C:** *Flag Arguments* são realmente ruins, mas a solução proposta na alternativa (remover os outros parâmetros) é absurda e não faz sentido técnico.
 - **D:** O problema do `println` existe, mas a solução de usar `switch` não resolve a complexidade da função.
 - **E:** Esta alternativa defende tudo o que o Clean Code abomina (nomes curtos e o erro técnico do `==` em Java para Strings).

Questões

🔗 #326847 FGV - 2014 - Analista em Tecnologia da Informação e Comunicação (PROCEMPA)/Analista em Programação e Suporte de Sist... ✖

Definir a responsabilidade de cada classe é um aspecto muito importante que deve ser observado durante a modelagem de um projeto de sistema de software.

Em relação aos princípios essenciais de boas práticas de modelagem orientada a objeto assinale a afirmativa correta.

- ☐ A O encapsulamento e a adoção de interfaces permitem diminuir o acoplamento e a coesão entre classes.
- ☐ B Cada classe deve assumir uma única responsabilidade bem definida como meio de aumentar a coesão e assegurar o encapsulamento.
- ☐ C As responsabilidades de uma classe devem estar relacionadas entre si para facilitar o entendimento e aumentar as chances de sua reutilização.
- ☐ D Para promover a flexibilidade e a facilidade de manutenção a longo prazo, as classes devem ser fortemente acopladas e encapsuladas.
- ☐ E Minimizar o acoplamento implica em classes com alta coesão.

Questões

🔗 #326847 FGV - 2014 - Analista em Tecnologia da Informação e Comunicação (PROCEMPA)/Analista em Programação e Suporte de Sist... ✖

Definir a responsabilidade de cada classe é um aspecto muito importante que deve ser observado durante a modelagem de um projeto de sistema de software.

Em relação aos princípios essenciais de boas práticas de modelagem orientada a objeto assinale a afirmativa correta.

- ☐ A O encapsulamento e a adoção de interfaces permitem diminuir o acoplamento e a coesão entre classes.
- ☐ B Cada classe deve assumir uma única responsabilidade bem definida como meio de aumentar a coesão e assegurar o encapsulamento.
- ☒ C As responsabilidades de uma classe devem estar relacionadas entre si para facilitar o entendimento e aumentar as chances de sua reutilização.
- ☐ D Para promover a flexibilidade e a facilidade de manutenção a longo prazo, as classes devem ser fortemente acopladas e encapsuladas.
- ☐ E Minimizar o acoplamento implica em classes com alta coesão.

Questões

🔗 #3722452 FUNDATEC - 2025 - Professor (Pref Três de Maio)/Linguagem Computacional ✖

Ao revisar um trecho de código de um colega, um professor de linguagem computacional se depara com a seguinte situação:

```
def process_data(input):  
    data = []  
    for i in range(len(input)):  
        if input[i] > 0:  
            data.append(input[i] * 2)  
    total = 0  
    for x in data: total += x  
    avg = total / len(data) if data else 0  
    result = []  
    for val in data:  
        if val > avg: result.append(val)  
    return result
```

Considerando as boas práticas de programação, qual alternativa descreve a melhor abordagem para refatorar esse código?

- ☐ A Apenas melhorar os nomes das variáveis e adicionar comentários explicando cada seção do código.
- ☐ B Manter a estrutura atual, mas adicionar tratamento de exceções para divisão por zero no cálculo da média.
- ☐ C Combinar todos os loops em um único loop para melhorar a eficiência, mesmo que isso torne o código mais complexo.
- ☐ D Converter o código para uma linguagem de programação mais eficiente, como C++, em vez de modificar a versão em Python.
- ☐ E Dividir o código em funções especializadas com nomes descritivos, substituir loops complexos por compreensões de lista e adicionar um docstring.

Questões

🔗 #3722452 FUNDATEC - 2025 - Professor (Pref Três de Maio)/Linguagem Computacional ✖

Ao revisar um trecho de código de um colega, um professor de linguagem computacional se depara com a seguinte situação:

```
def process_data(input):  
    data = []  
    for i in range(len(input)):  
        if input[i] > 0:  
            data.append(input[i] * 2)  
    total = 0  
    for x in data: total += x  
    avg = total / len(data) if data else 0  
    result = []  
    for val in data:  
        if val > avg: result.append(val)  
    return result
```

Considerando as boas práticas de programação, qual alternativa descreve a melhor abordagem para refatorar esse código?

- ☐ A Apenas melhorar os nomes das variáveis e adicionar comentários explicando cada seção do código.
- ☐ B Manter a estrutura atual, mas adicionar tratamento de exceções para divisão por zero no cálculo da média.
- ☐ C Combinar todos os loops em um único loop para melhorar a eficiência, mesmo que isso torne o código mais complexo.
- ☐ D Converter o código para uma linguagem de programação mais eficiente, como C++, em vez de modificar a versão em Python.
- ☒ E Dividir o código em funções especializadas com nomes descritivos, substituir loops complexos por compreensões de lista e adicionar um docstring.

Questões

🔗 #3029293 Instituto Consulplan - 2024 - Analista Judiciário (TJ MA)/Analista de Sistemas/Desenvolvimento ✖

Determinado desenvolvedor de softwares, com experiência em uma linguagem de programação específica, foi contratado por uma *Software House* de grande porte para trabalhar com a mesma linguagem de programação do seu domínio. Durante o processo de *onboarding*, o profissional recebeu diversas orientações sobre os produtos da empresa, as necessidades e também os desafios de cada um dos projetos; na parte técnica, o profissional foi instruído inúmeras vezes que na organização existem diversos códigos legados, que foram produzidos no início desses projetos por desenvolvedores que já não fazem mais parte da organização e que, por norma de um acordo interno, todas as intervenções nesses códigos necessitam passar por um processo de refatoração e adequação de boas práticas de programação. Como material de apoio, o desenvolvedor recebeu toda a documentação interna da empresa onde constam os padrões de código, baseados em técnicas de Clean Code, que devem ser seguidos nas implementações, além de uma indicação de leitura e estudo de livros relacionados a esse tema. Diante do exposto, assinale a afirmativa INCORRETA.

- ☐ A É importante aplicar o princípio da responsabilidade exclusiva que afirma que uma classe deve ter um, e apenas um, motivo para mudar.
- ☐ B Para confirmar se uma função faz apenas uma “única coisa”, pode-se verificar se todas as instruções internas estão no mesmo nível de abstração.
- ☐ C É recomendável a criação de funções com finalidade específica, pequenas e devidamente aninhadas, facilitando o processo de leitura e compreensão.
- ☐ D É aconselhável a utilização de nomes de variáveis passíveis de busca, que não possuam apenas uma letra ou números, pois são de difícil localização em um texto longo.

Questões

🔗 #3029293 Instituto Consulplan - 2024 - Analista Judiciário (TJ MA)/Analista de Sistemas/Desenvolvimento ✖

Determinado desenvolvedor de softwares, com experiência em uma linguagem de programação específica, foi contratado por uma *Software House* de grande porte para trabalhar com a mesma linguagem de programação do seu domínio. Durante o processo de *onboarding*, o profissional recebeu diversas orientações sobre os produtos da empresa, as necessidades e também os desafios de cada um dos projetos; na parte técnica, o profissional foi instruído inúmeras vezes que na organização existem diversos códigos legados, que foram produzidos no início desses projetos por desenvolvedores que já não fazem mais parte da organização e que, por norma de um acordo interno, todas as intervenções nesses códigos necessitam passar por um processo de refatoração e adequação de boas práticas de programação. Como material de apoio, o desenvolvedor recebeu toda a documentação interna da empresa onde constam os padrões de código, baseados em técnicas de Clean Code, que devem ser seguidos nas implementações, além de uma indicação de leitura e estudo de livros relacionados a esse tema. Diante do exposto, assinale a afirmativa INCORRETA.

- ☒ A É importante aplicar o princípio da responsabilidade exclusiva que afirma que uma classe deve ter um, e apenas um, motivo para mudar.
- ☐ B Para confirmar se uma função faz apenas uma “única coisa”, pode-se verificar se todas as instruções internas estão no mesmo nível de abstração.
- ☐ C É recomendável a criação de funções com finalidade específica, pequenas e devidamente aninhadas, facilitando o processo de leitura e compreensão.
- ☐ D É aconselhável a utilização de nomes de variáveis passíveis de busca, que não possuam apenas uma letra ou números, pois são de difícil localização em um texto longo.

Questões

 #3597803 CEBRASPE (CESPE) - 2025 - Analista Judiciário (TJ PA)/Análise de Sistemas ✖



A respeito de criptografia, do *clean code* e das técnicas de refactoring, julgue o item seguinte.

Um código limpo ou *clean code* deve priorizar legibilidade semântica e baixo ruído de manutenção, devendo comentários ser usados apenas quando a intenção da lógica não puder ser claramente expressa pelo código em si.

☐ C Certo

☐ E Errado

Questões

🔗 #3597803 CEBRASPE (CESPE) - 2025 - Analista Judiciário (TJ PA)/Análise de Sistemas ✖



A respeito de criptografia, do *clean code* e das técnicas de refactoring, julgue o item seguinte.

Um código limpo ou *clean code* deve priorizar legibilidade semântica e baixo ruído de manutenção, devendo comentários ser usados apenas quando a intenção da lógica não puder ser claramente expressa pelo código em si.

☒ Certo

☐ Errado

Questões

🔗 #3284936 FCPC - 2025 - Analista de Tecnologia da Informação (UFC)/Arquitetura e Desenvolvimento de Sist... ✖



Em relação ao conceito de Clean Code, qual das alternativas abaixo apresenta corretamente uma combinação de princípios fundamentais?

- ☐ A O Código deve ser fácil de entender, com nomes claros e descritivos para variáveis, funções e classes.
- ☐ B O código deve ser otimizado ao máximo, mesmo que isso reduza sua legibilidade, para garantir melhor desempenho.
- ☐ C O código deve ser escrito de forma que seja fácil de testar, mantendo códigos mortos comentados para auxiliar no entendimento.
- ☐ D Cada função ou classe deve ter uma única responsabilidade e seus nomes devem refletir claramente o que elas fazem, mesmo que aumente a repetição de código.

Questões

🔗 #3284936 FCPC - 2025 - Analista de Tecnologia da Informação (UFC)/Arquitetura e Desenvolvimento de Sist... ✖



Em relação ao conceito de Clean Code, qual das alternativas abaixo apresenta corretamente uma combinação de princípios fundamentais?

- ☒ A O Código deve ser fácil de entender, com nomes claros e descritivos para variáveis, funções e classes.
- ☐ B O código deve ser otimizado ao máximo, mesmo que isso reduza sua legibilidade, para garantir melhor desempenho.
- ☐ C O código deve ser escrito de forma que seja fácil de testar, mantendo códigos mortos comentados para auxiliar no entendimento.
- ☐ D Cada função ou classe deve ter uma única responsabilidade e seus nomes devem refletir claramente o que elas fazem, mesmo que aumente a repetição de código.

Questões

🔗 #3251237 CEBRASPE (CESPE) - 2025 - Analista Judiciário (TRF 6ª Região)/Apoio Especializado/Análise de Sistemas de Informação ✖

Acerca das técnicas de *Clean Code* e de *Refactoring*, julgue o item a seguir.

Na declaração de variáveis, de acordo com as técnicas de *Clean Code*, devem ser utilizados nomes e funções curtos e abstratos, desde que seu significado esteja descrito nos comentários ao lado do código.

☐ C Certo

☐ E Errado

Questões

🔗 #3251237 CEBRASPE (CESPE) - 2025 - Analista Judiciário (TRF 6ª Região)/Apoio Especializado/Análise de Sistemas de Informação ✖

Acerca das técnicas de *Clean Code* e de *Refactoring*, julgue o item a seguir.

Na declaração de variáveis, de acordo com as técnicas de *Clean Code*, devem ser utilizados nomes e funções curtos e abstratos, desde que seu significado esteja descrito nos comentários ao lado do código.

☐ Certo

☒ Errado